



How to Simulate Vehicle Dynamics

This page will go through the steps of simulating a vehicle's movement, enabling for example:

- Validation of control features, simulating a vehicle's position over time.
- Game Design: Output a vehicle's position depending on user input.

If you are instead looking for a tool that can already simulate dynamics, please see [Tools](#). This guide assumes the vehicle a single rigid body, not split into separate moving or flexible parts (See [Multi-Body Simulation](#)).

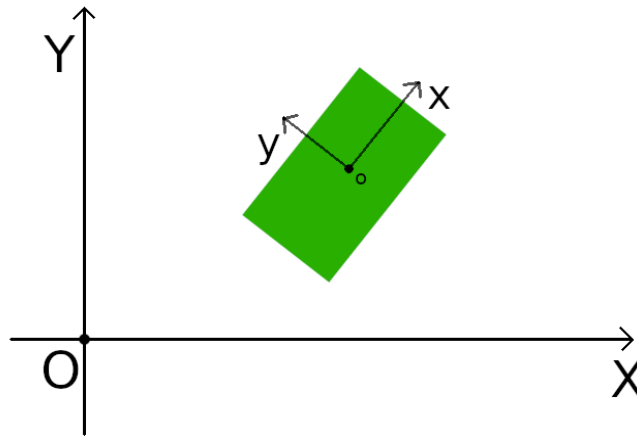
- [Set Coordinate Systems](#)
- [Decide Degrees of Freedom](#)
- [How to get Translation and Rotation](#)
- [System of Equations](#)
- [Model Forces & Torques](#)
- [Perform Coordinate Transformations](#)
- [Solvers](#)
 - [Numerically Integrate for Translation and Rotation](#)
 - [ODE – Ordinary Differential Equations](#)
 - [Solving system of differential equations](#)
- [Visualization & Analysis](#)

Set Coordinate Systems

To simulate the vehicle we need to calculate the position and rotation in relation to a coordinate system.

There are many types of coordinate systems, and the best one will depend on your use-case.

We will focus on Cartesian coordinate systems ([Wikipedia](#)) with 90degree angles between axes, and start with showing a X-Y-coordinate system.



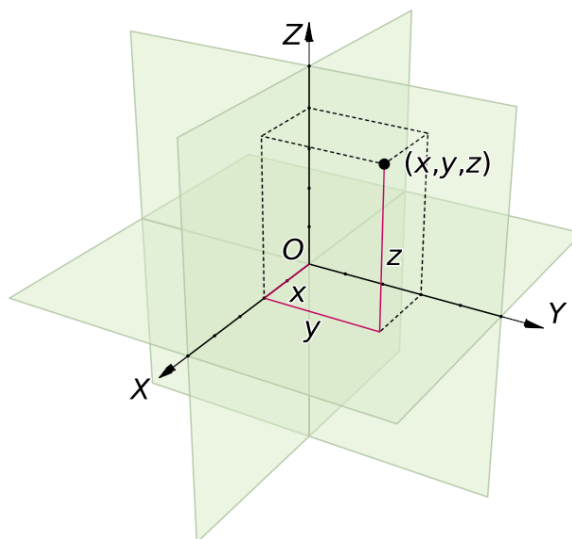
In general you will have a global “earth fixed” coordinate system, centred at origin (O). In above picture axes are denoted by capital X and Y .

Additionally there might be a need to have local coordinate systems fixed to the simulated vehicles, see x - y axes in above picture.

The number of local coordinate systems on a single vehicle will depend on how advanced your simulation model is.

If you simulate your whole vehicle as a single rigid body, then one local coordinate system will suffice.

For three dimensional simulations we will use the common right handed X - Y - Z coordinate system (Right handed referring to z axis pointing upwards: [Wikipedia](https://en.wikipedia.org/wiki/Right-handed_coordinate_system)). For more reading about coordinate systems, see: [Coordinate Systems](https://en.wikipedia.org/wiki/Coordinate_systems). In the coming chapters we will assume a X - Y - Z coordinate system is used, but the principals will be the same as for a flat X - Y coordinate system.



Decide Degrees of Freedom

When setting up your simulation you will decide degrees of freedom (DOF) depending on your use-case and how realistic the simulation model needs to be.

Assuming a rigid body, movement along each axis in a X-Y-Z coordinate system gives 3 DOF while rotation about those axes give 3 additional possible DOF (*roll, pitch & yaw*). It is crucial to not over-complicate your simulation and only to consider enough DOF to cover your use-case.

Example of use-cases and suitable amount of DOF:

Simulate a cart's movement along a straight rail	1 DOF (<i>X</i>)
– Simple game with a birds-eye-view of a ground vehicle in an environment without elevation (movement in z axis). – Kinematic bicycle model (Used for simple dynamics in many fields such as automotive)	3 DOF (<i>X, Y, yaw</i>)
Aircraft or driving simulator	6 DOF (<i>X, Y, Z, roll, pitch, yaw</i>)

How to get Translation and Rotation

We want to calculate translation (position) along and rotation around the axes corresponding to the designated degrees of freedom. If we for example know the translation along *X, Y, Z*, we know where the vehicle is located in 3D space. In order to calculate the translation and rotation, we will have to work backwards from the translational and rotational acceleration.

Lets first look at movement along a single axis. With Newton's Second Law we can get the relation between forces (*F*) applied to the vehicle and the resulting translational acceleration (*a*), assuming mass (*m*) stays constant ([Wikipedia](#)).

$$F = ma$$

$$Units : [N] = [kg][m/s^2]$$

The corresponding relation between applied torque (τ) around an axis and resulting rotational acceleration (*a*) around that axis is as below, assuming inertia (*I*) stays constant. For more reading: [Wikipedia](#).

$$\tau = I\alpha$$

$$Units : [Nm] = [kgm^2][rad/s^2]$$

If we “keep track” of all the calculated accelerations over time in our simulation (Through numerical integration, explained later), we will learn the following of the vehicle for any given time:

$$Acceleration, \quad a \quad [m/s^2]$$

$$Velocity, \quad v \quad [m/s]$$

$$Translation, \quad s \quad [m]$$

$$Angular Acceleration, \quad \alpha \quad [rad/s^2]$$

$$Angular Velocity, \quad \omega \quad [rad/s]$$

$$Rotation, \quad \theta \quad [rad]$$

System of Equations

For a system of 6 degrees of freedom, we have the following:

$$\begin{bmatrix} F_X \\ F_Y \\ F_Z \\ \tau_X \\ \tau_Y \\ \tau_Z \end{bmatrix} = \begin{bmatrix} \text{Mass Matrix} \\ 6 \times 6 \end{bmatrix} \times \begin{bmatrix} a_X \\ a_Y \\ a_Z \\ \alpha_X \\ \alpha_Y \\ \alpha_Z \end{bmatrix}$$

or

$$\begin{bmatrix} a_X \\ a_Y \\ a_Z \\ \alpha_X \\ \alpha_Y \\ \alpha_Z \end{bmatrix} = \begin{bmatrix} \text{Mass Matrix} \\ 6 \times 6 \end{bmatrix}^{-1} \times \begin{bmatrix} F_X \\ F_Y \\ F_Z \\ \tau_X \\ \tau_Y \\ \tau_Z \end{bmatrix}$$

Where the [Mass Matrix](#) is:

$$Mass\ Matrix = \begin{bmatrix} m & & & & \\ & m & & & \\ & & m & & \\ & & & I_X & \\ & & & & I_Y \\ & & & & & I_Z \end{bmatrix} + \begin{bmatrix} Added \\ Mass/Inertia \\ Coefficients \end{bmatrix}$$

If the vehicle traverses a heavier medium such as water, both the mass (m) and inertia (I) should factor in the displaced mass ([Added Mass/Inertia Coefficient](#)). Added mass refers to the additional forces and moments exerted on an accelerating body by the surrounding medium. Whenever a body accelerates in any direction, it must also cause the fluid it moves through to accelerate, resulting in the need for extra forces and moments compared to a situation where the body moves in a vacuum.

Model Forces & Torques

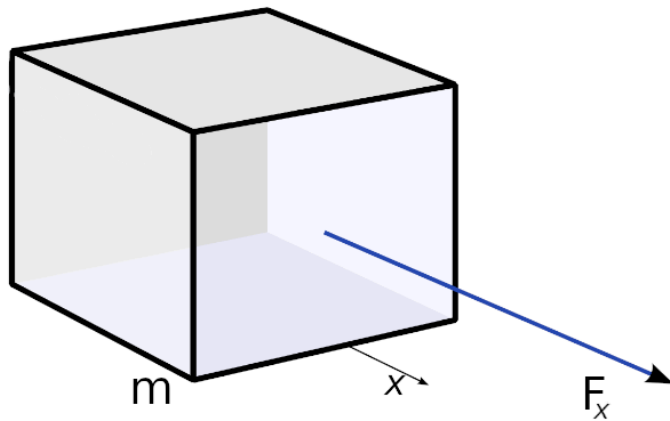
Depending on vehicle and scope of simulation, the forces you decide to apply to the vehicle will differ.

Remember that all the forces and torques will be summarized together in respect to global coordinate system X-Y-Z, and the resulting total force and torque will determine translational & rotational acceleration in respect to X-Y-Z. That being said, some of the forces you model on your vehicle will be calculated with respect to the vehicle's local coordinate system x-y-z. The next chapter will explain how local forces in x-y-z can be transformed into forces in global X-Y-Z.

$$\begin{aligned} Local\ Forces &: [F_x, F_y, F_z] \\ Global\ Forces &: [F_X, F_Y, F_Z] \end{aligned}$$

A force applied in positive x direction, will result in a positive acceleration in x

$$\begin{aligned} F_x &= ma_x \\ a_x &= \frac{F_x}{m} \end{aligned}$$



Lets have a look at different types of forces.

Constant force

Example: Gravitational force at surface of earth.

$$F_z = ma_z = -mg$$

Time dependent force

Example: A local propulsion force in the vehicles local x-direction that increases linearly with time (t , unit: seconds).

$$F_x(t) = 3t$$

Position dependent force

Example: Spring force which gets linearly larger with displacement from equilibrium ([Wikipedia](https://en.wikipedia.org/wiki/Hooke%27s_law)).

$$F_z(z) = -kz$$

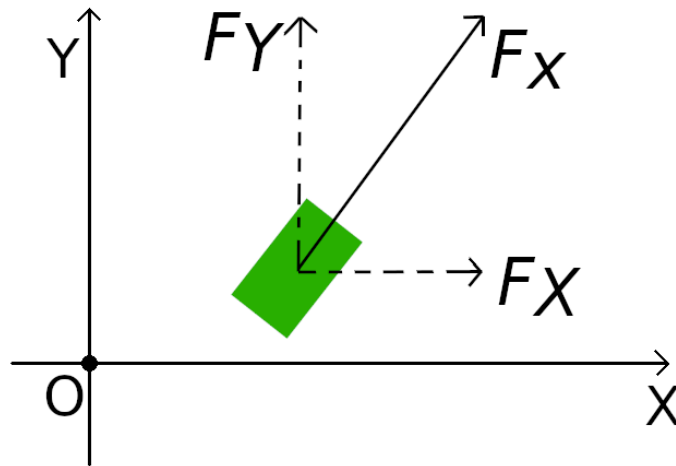
Velocity dependent force

Example: Drag force acting in opposite direction of velocity (Simplified, see: [Wikipedia](https://en.wikipedia.org/wiki/Fluid_drag)).

$$F_x(v_x) = -Cv_x^2$$

Perform Coordinate Transformations

A vehicle's accelerations (and resulting velocities) are either calculated in local or global coordinate system. We will select to calculate in local since many forces applied to a vehicle are calculated in reference to the vehicle's local coordinate systems (x-y-z). Forward propulsion could for example be a force in the positive local x axis. When calculating the vehicle's accelerations in local, we need to convert any forces/torques acting in global into local. One example being gravity which works in negative Z direction for most three dimensional simulations.



When keeping track of the vehicle's position in the time domain, local accelerations and velocities need to be converted into the global coordinate system (X-Y-Z), a rotation matrix R is used to convert to the global coordinate system ([Wikipedia](https://en.wikipedia.org/wiki/Rotation_matrix)). R is commonly used for transforming forces, velocities & accelerations.

$$\begin{aligned}\vec{F}_{Global} &= R \times \vec{F}_{Local} \\ \begin{bmatrix} F_X \\ F_Y \\ F_Z \end{bmatrix} &= R \times \begin{bmatrix} F_x \\ F_y \\ F_z \end{bmatrix}\end{aligned}$$

Similarly the reverse transformation is done with the inverse of R .

$$\vec{F}_{Local} = R^{-1} \times \vec{F}_{Global}$$

If we for example keep track of the vehicle's velocity in a global perspective, we might still be interested in how fast the vehicle is moving forwards (local x direction). To get this value, we would need to transform the global velocity vector into a local vector.

$$\vec{v}_{Local} = R^{-1} \times \vec{v}_{Global}$$

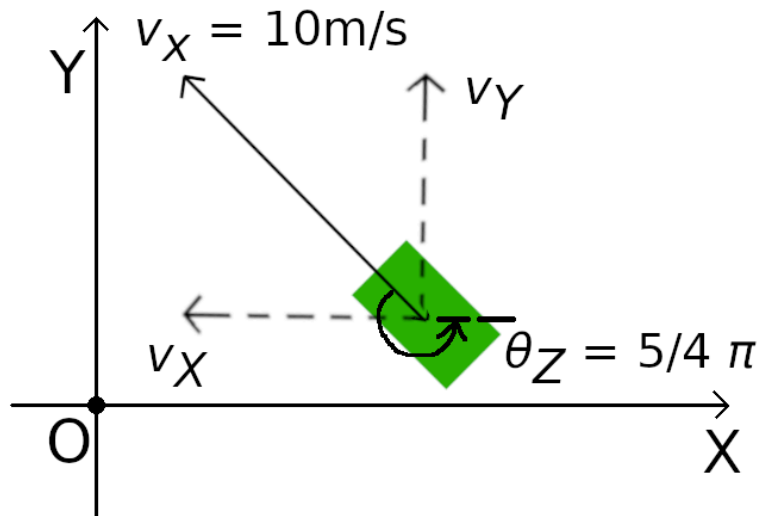
Rotation matrices describe rotations about the origin. The rotation matrix R for 3 dimensional rotation is built with 3 rotation matrices, one for each angle of rotation.

$$R = R_x(\theta_x) \times R_y(\theta_y) \times R_z(\theta_z)$$

The rotation matrix R corresponding to a left angle rotation θ around the z axis will be below (Yaw). Depending on translational Degree of Freedom, R dimensions will differ. Note: This is valid for a [right-hand](#) system.

$$R_z(\theta_z) = \begin{bmatrix} \cos \theta_z & -\sin \theta_z \\ \sin \theta_z & \cos \theta_z \end{bmatrix} \quad R_z(\theta_z) = \begin{bmatrix} \cos \theta_z & -\sin \theta_z & 0 \\ \sin \theta_z & \cos \theta_z & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

If there is no translational freedom in Z direction, a yaw rotation around Z axis might look like this:



$$\begin{aligned} \vec{v}_{Global} &= R_z \times \vec{v}_{Local} \\ \begin{bmatrix} v_X \\ v_Y \end{bmatrix} &= \begin{bmatrix} \cos \theta_z & -\sin \theta_z \\ \sin \theta_z & \cos \theta_z \end{bmatrix} \times \begin{bmatrix} v_x \\ v_y \end{bmatrix} \\ &= \begin{bmatrix} \cos \frac{5\pi}{4} & -\sin \frac{5\pi}{4} \\ \sin \frac{5\pi}{4} & \cos \frac{5\pi}{4} \end{bmatrix} \times \begin{bmatrix} 10 \\ 0 \end{bmatrix} m/s \\ &= \begin{bmatrix} -7.071 \\ 7.071 \end{bmatrix} m/s \end{aligned}$$

For more examples of coordinate transformations: [Coordinate Transformations](#)

Solvers

Numerically Integrate for Translation and Rotation

In previous chapters we mentioned we would “keep track” of the accelerations to calculate resulting position and rotation, now we will examine the different methods to do so. It is crucial to know the relation between the accelerations and translation/rotation:

translational acceleration (a) is the second derivative of translation (s) with respect to time (t).

rotational acceleration (α) is the second derivative of rotation (θ) with respect to time (t).

In other words, the rate of change of s is v , the rate of change of v is a .

$$a(t) = \dot{v}(t) = \ddot{s}(t)$$

$$a = \frac{d^2 s}{dt^2} = \ddot{s}$$

$$v = \frac{ds}{dt} = \dot{s}$$

$$\alpha(t) = \dot{\omega}(t) = \ddot{\theta}(t)$$

$$\alpha = \frac{d^2 \theta}{dt^2} = \ddot{\theta}$$

$$\omega = \frac{d\theta}{dt} = \dot{\theta}$$

The velocities v and ω for the respective degrees of freedom are to be integrated over time to calculate how the vehicle is moving.

Doing so and logging the data will make it possible to get the trajectory data as a function of time for the vehicle.

As long we have non-constant accelerations, there will be a need to numerically integrate instead of integrating analytically (Forces applied to a vehicle are in most cases not constant). [Numerical methods](#) are commonly used for solving differential equations for the unknown position of the body as a function of time.

Goal: Numerically integrate translational/rotational velocities to get a vehicle's translation(position) & rotation

Lets looks at how velocity (v) & translation (s) can be numerically integrated between time t_0 & t_1 from acceleration (a) along an axis.

$$\Delta t = t_1 - t_0$$

$$v_{t1} = v_{t0} + \Delta t \cdot a_{t0}$$

$$s_{t1} = s_{t0} + \Delta t \cdot v_{t0}$$

This is the Euler method. Euler's method is crude but fast, there are many available methods for numerically integrating, where the best method will depend on your preferred balance between accuracy & computational speed. More accurate methods, such as the Runge-Kutta method, use multiple function evaluations per time step to reduce the error. See [Wikipedia](https://en.wikipedia.org/wiki/Numerical_integration) for further breakdown of the different methods of numerical integration.

In the above numerical integrations, it will be necessary to first get the acceleration (a) to calculate velocity and translation for each time sample.

If you rearrange Newton's second law it can be written as the below equation. Lets look at X axis for starters.

$$\begin{aligned} a_X &= \frac{F_X}{m} \\ \frac{dv_X}{dt} &= \frac{1}{m} F_X \\ \frac{d^2 s_X}{dt^2} &= \frac{1}{m} (F_{X,1} + F_{X,2} + \dots + F_{X,n}) \end{aligned}$$

This turns into a *ordinary differential equation* (ODE) if any of the forces (F) are a function of translation or velocity (e.g. wind resistance) in x. With differential equation we mean an equation involving derivatives of a function or functions.

Similarly for rotation around X axis:

$$\begin{aligned} \alpha_X &= \frac{\tau_X}{I} \\ \frac{d\omega_X}{dt} &= \frac{1}{I} \tau_X \\ \frac{d^2 \theta_X}{dt^2} &= \frac{1}{I} (\tau_{X,1} + \tau_{X,2} + \dots + \tau_{X,n}) \end{aligned}$$

For more reading on differential equations: [Wikipedia](#)

ODE – Ordinary Differential Equations

An *ordinary differential equation* (ODE) contains one or more derivatives of a dependent variable. The *order* of the ODE is equal to the highest-order derivative of the dependent variable that appears in the equation. We deal with accelerations which is the second derivative of time, meaning we have second order ODE. The term “*ordinary*” is used in contrast with the term [partial differential equation](#), which may be with respect to *more than one* independent variable.

In an *initial value problem*, the ODE is solved by starting from an initial state. Using the initial condition for position, rotation, velocities and rotational velocities, as well as a period of time over which the answer is to be obtained, (t_0, t_n) , the solution is obtained iteratively. At each step the solver applies the chosen algorithm (e.g. Euler) to the results of previous steps. At the first such step, the initial condition provides the necessary information that allows the integration to proceed. The final result is that the ODE solver returns a vector of time steps $t=[t_0, t_1, t_2, \dots, t_n]$ as well as the corresponding solution at each step, e.g. $X=[X_0, X_1, X_2, \dots, X_n]$.

Here is a simple overview example of how the Euler method can be used to numerically integrate the position/velocity of a vehicle along global X axis:

1. Choose a time step, Δt .
2. Initialize the position and velocity of the vehicle, X_0 & v_{X0} , to their initial values.
3. Loop over time steps from t_0 to t , incrementing time by Δt at each step. At each time step:
 - Calculate resulting acceleration $a_X(t+\Delta t)$
 - Get the velocity of the vehicle using Euler: $v_X(t+\Delta t) = V(t) + a_X(t)\Delta t$.
 - Get the position of the vehicle using Euler: $X(t+\Delta t) = X(t) + v_X(t)\Delta t$.
4. When you reach the desired time t , the position of the vehicle is $X(t)$.

Solving system of differential equations

We now need to numerically integrate position/velocity for each degree of freedom of our model.

Depending on simulation tool, the needed input equation format will differ. Numerical

integration might be easier to perform in either local or global coordinate system (x/X etc.).

It is common to have to rewrite our second order ODEs as first order ODE.

Let's look at an simplified example with air resistance force applied in local x axis:

$$a_x = \frac{F_x}{m}$$

$$\ddot{s}_x = \frac{-Cv_x^2}{m} = \frac{-C(\dot{s}_x)^2}{m}$$

With initial conditions:

$$s_x(t_0) = s_{x,0}$$

$$\dot{s}_x(t_0) = \dot{s}_{x,0}$$

We now introduce the variables:

$$x_1 = s_x$$

$$x_2 = \dot{s}_x$$

The necessary set of differential equations for x_1' and x_2' become:

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = \frac{-Cx_2^2}{m}$$

with similar initial conditions:

$$x_1(t_0) = s_{x,0}$$

$$x_2(t_0) = \dot{s}_{x,0}$$

Visualization & Analysis

For game design the goal is to visualize the position/rotation of the vehicle in real time after each numerical step of simulation.

For other use-cases common in engineering it might be important to store and analyze the output of each numerical step of the simulation.

Find tools to facilitate your vehicle simulation here: [Tools](#)
